LogRMS: A Two-Byte Optimizer with TinyStories Training Results

Matthew Belcher
Computer Science, Open Institute of Technology (OPIT), Malta
email to be added

Evidence update C — 13 July 2026

Abstract

Adaptive optimizers often store several values for every model parameter. LogRMS uses exactly two persistent tensor bytes per parameter: an int8 log-encoded second moment and a uint8 record of recent nonzero gradient signs. It computes the second-moment recurrence and current update in FP32, then uses stochastic rounding to store the moment for the next step. Sign history only reduces updates on coordinates whose gradients alternate frequently. Compared with ordinary FP32-state AdamW, the persistent tensor state is 75% smaller.

A controlled 49,515,008-parameter TinyStories systems benchmark processed 471,765,959 training tokens on one NVIDIA L4. Compiled LogRMS achieved a median full-step time of 107.369 ms versus 107.263 ms for fused AdamW, a 0.10% difference, while using 2.0000 versus 8.0000 persistent state bytes per parameter. Its total wall time was 3.5% longer, mainly because of a 207-second one-time compilation. Under the shared, untuned schedule, LogRMS finished at test perplexity 3.7698 versus 3.6911 for fused AdamW, a 2.13% gap. Peak live PyTorch allocation was 12.7% lower, although that number covers the whole training process rather than the optimizer alone. A separate 5.06M-parameter run and small diagnostic tests show that the same optimizer can train reliably, but can struggle on curved objectives and rare gradient spikes. These results support LogRMS as a practical low-state optimizer candidate. They do not show that it always converges as well as AdamW.

1 Introduction

Optimizer state can use a large share of training memory. Ordinary Adam maintains exponential moving averages of both gradients and squared gradients [9]; in the common FP32-state form these two tensors require approximately eight bytes per parameter, before weights, gradients, master copies, or framework metadata. At large model sizes, a few bytes per parameter add up quickly.

Quantized optimizers reduce this burden by compressing Adam's moments [5], while factorized and structured approaches reduce the number of stored statistics [11, 1]. LogRMS stores one coordinate-wise RMS statistic in one byte and uses a second byte for short sign history instead of keeping an Adam-style first moment.

This paper provides:

1. an exact two-byte per-parameter persistent tensor layout;
2. stochastic log encoding that preserves small changes in the stored second moment;
3. a standard `torch.optim.Optimizer` implementation with normal checkpoint support;
4. a `torch.compile` execution path; and
5. transformer results, timing measurements, exact state accounting, and diagnostic tests.

The results show that LogRMS can train 5.06M- and 49.5M-parameter transformers. On the 49.5M model, its compiled implementation had almost the same steady-state full-step time as fused AdamW. The experiments use one final-run seed at each scale, so they do not establish general convergence parity, distributed scaling, or a universal peak-memory advantage.

2 Related work

AdaGrad established per-coordinate adaptive scaling from accumulated squared gradients [6]. Adam combines first- and second-moment exponential averages with bias correction [9]; AdamW separates weight decay from the loss-gradient update [10]. LogRMS keeps Adam's second-moment form and bias correction but removes the first moment.

Memory-efficient alternatives include Adafactor's factored second moments [11], SM3's covering-set accumulators [1], and block-wise 8-bit quantization of optimizer state [5]. LogRMS keeps a simple coordinate-wise state for tensors of any rank. Its two bytes are not necessarily the smallest possible representation when factorization or block metadata is considered.

Sign-based methods demonstrate that gradient direction can remain useful after discarding magnitude [3]; Lion uses sign information in an update constructed from momentum-like state [4]. LogRMS does not use a sign-only update. It retains gradient magnitude through RMS

normalization and uses sign history only to reduce selected updates. Stochastic rounding is commonly used with limited-precision arithmetic [8]; here it is applied to the stored second moment.

3 Method

3.1 RMS update without a first moment

For parameter θ, gradient g_t, and second-moment decay β_2, the optimizer computes

$$v_t = \beta_2 \widetilde{v}_{t-1} + (1-\beta_2) g_t^2, \tag{1}$$

$$\widehat{v}_t = \frac{v_t}{1-\beta_2^t}, \tag{2}$$

$$\theta'_t = (1-\eta\lambda)\theta_{t-1}, \tag{3}$$

$$\theta_t = \theta'_t - \eta c_t \frac{g_t}{\sqrt{\widehat{v}_t}+\epsilon}, \tag{4}$$

where η is the learning rate, λ is decoupled weight decay, c_t is sign confidence, and $\widetilde{v}_{t-1}$ is the decoded stored state. All quantities in the recurrence and normalization are calculated in FP32. The fresh, unquantized v_t is used for the current update; compression affects the next recurrence.

The state encoding does not limit model precision. LogRMS can update FP32 parameters because it keeps the current gradient and moment calculation in FP32. Only the stored second moment is compressed for the next step.

3.2 One-byte logarithmic second moment

An int8 code $q = -128$ denotes exact zero. Other codes decode as

$$\mathcal{D}(q) = 2^{q/2}, \qquad q \in \{-127, \ldots, 127\}. \tag{5}$$

The half-octave spacing covers approximately $2^{-63.5}$ to $2^{63.5}$. Given adjacent represented values $a \le v_t \le b$, encoding selects b with probability

$$p_b = \frac{v_t - a}{b - a} \tag{6}$$

and a otherwise. Consequently, $\mathbb{E}[\mathcal{D}(q_t)] = v_t$ away from range clipping. A stateless integer hash of coordinate, parameter ordinal, step, and seed supplies the random value without consuming model RNG state or storing a residual. Stochastic rounding prevents small EMA changes from being rounded away repeatedly.

3.3 One-byte sign history

A uint8 variable-length prefix code stores zero through seven prior nonzero signs. The current nonzero sign completes an eight-sign sequence. If T_t is the number of adjacent transitions among those eight signs, the confidence is

$$c_t = 1 - 0.8\frac{T_t}{7}, \qquad T_t \in \{0, \ldots, 7\}. \tag{7}$$

Table 1: Persistent tensor state in the 49.5M systems benchmark. Measured 8-bit AdamW includes its tensor metadata in the tested torchao configuration.

Optimizer	B/parameter	Reduction vs. 8 B
AdamW, FP32 moments	8.0000	—
torchao AdamW 8-bit	2.0460	74.4%
LogRMS log8	**2.0000**	**75.0%**

Table 2: Aggregate state payload implied by exact 8 and 2 B/parameter layouts.

Parameters	AdamW (GiB)	LogRMS (GiB)	Saving (GiB)
1B	7.45	1.86	5.59
7B	52.15	13.04	39.12
13B	96.86	24.21	72.64
70B	521.54	130.39	391.16
1T	7,450.58	1,862.65	5,587.94

Thus $c_t \in [0.2, 1]$: stable signs receive the ordinary RMS update and frequently alternating signs are damped. Confidence is exactly one until eight genuine nonzero observations exist. A zero gradient neither advances history nor counts as a negative sign.

The implementation uses small cached lookup tensors to decode the history and map each 8-bit sequence to confidence.

3.4 Memory accounting

Table 1 reports unique persistent tensor storage held by each optimizer. Python objects, allocator rounding, and non-tensor metadata are not included. At one trillion parameters, the difference between two and eight bytes is about 5.46 TiB in total. Sharding divides that saving among workers.

This number is different from CUDA allocator measurements. `max_memory_allocated` records the peak live tensor allocation across the complete PyTorch process, including parameters, gradients, activations, optimizer state, temporaries, compiler buffers, and evaluation. `max_memory_reserved` also reflects blocks retained by the caching allocator. Neither measurement isolates optimizer memory, and both may omit allocations made outside the PyTorch allocator. We report them as whole-process measurements, not as optimizer-state measurements.

4 5.06M TinyStories convergence study

4.1 Protocol

We trained a 5,063,424-parameter causal transformer on the official TinyStories dataset [7]. Each final optimizer run saw exactly two complete training epochs: 943,531,918

tokens in 115,178 update steps. Runs used the same initial-state hash and minibatch order, BF16 autocast on an NVIDIA L4, decoupled weight decay 0.01, and global gradient clipping at 1.0.

Each optimizer received its own three-point learning-rate grid. AdamW and 8-bit AdamW used $\{3 \times 10^{-4}, 10^{-3}, 3{\times}10^{-3}\}$; LogRMS used $\{10^{-3}, 3{\times}10^{-3}, 10^{-2}\}$. A separate 20,004,864-token tuning budget selected 3×10^{-3} for all three. The unmatched LogRMS 10^{-2} point is a failed internal tuning point, not a comparative condition. The final comparison contains one seed.

4.2 Results

Table 3 gives post-training full-validation and held-out test metrics. LogRMS test loss was 0.0041 lower than AdamW (0.26% relative) and 0.0082 higher than 8-bit AdamW (0.54%). These gaps are small, but the experiment has only one final-run seed. It shows that LogRMS can train this model with two bytes of persistent state per parameter; it does not establish a reliable ranking.

The eager implementation launches many elementwise operations and allocates full-size FP32 temporaries. Its smaller persistent state therefore did not make it faster in this run. The difference between the tested 8-bit AdamW and LogRMS state payloads is only 2.93%, or 57.6 MiB per billion parameters.

5 Compilation and 5.06M parity

An execution-only prototype compiled the tensor group update with PyTorch Inductor [2] without changing the update rule. In a same-shape 5.06M-parameter optimizer microbenchmark, median step time fell from 85.50 ms eager to 4.645 ms compiled (18.4×), after 104 s of warm-up and compilation. Attempting to compile the entire Python `Optimizer.step` failed because Dynamo could not capture mutation of user-defined optimizer attributes; compiling the tensor group was the successful boundary.

A separate two-epoch TinyStories compiled run closely followed the eager reference run (Figure 2). Final full-validation loss was 1.5803 versus 1.5780, and test-loss difference was 0.0021. Across matched points, mean absolute validation-loss difference was 0.0031. The recorded end-to-end wall-time ratio was 4.48×. Because the runs were separate rather than a randomized same-process timing study, that ratio should be treated as an approximate acceleration result.

6 49.5M systems study

6.1 Protocol

The larger benchmark used a 49,515,008-parameter causal transformer with a 4096-token vocabulary, sequence length 256, hidden width 512, 8 attention heads, 15 pre-normalized blocks, a 2048-unit feed-forward width, tied input/output embeddings, and zero dropout. One complete official TinyStories training epoch contained 471,765,959 target tokens and 57,589 optimizer updates. Microbatch 16 with two-step accumulation produced 8,192 tokens per update.

All four runs used the same initial-state SHA-256, epoch permutation, FP32 model parameters and gradients, BF16 activations, global norm clipping at 1.0, decoupled weight decay 0.01, $\beta_2 = 0.999$, and $\epsilon = 10^{-8}$. The shared schedule warmed to a peak learning rate of 10^{-3} over 5% of updates and then decayed cosinusoidally to 1% of the peak. This common policy isolates implementation behavior; it is not independent optimizer tuning.

The four execution paths were PyTorch AdamW with CUDA foreach, PyTorch AdamW with `fused=True`, torchao 0.17.0 AdamW8bit, and compiled LogRMS. The environment was an NVIDIA L4, PyTorch 2.11.0+cu128, and CUDA 12.8. Median steady-state times exclude the first 20 updates. Total wall time includes model and optimizer construction, cold compilation, training, scheduled evaluation, CSV logging, checkpoints, and final full validation and test evaluation. The model itself was not compiled, so the benchmark measures differences in optimizer execution rather than model compilation.

6.2 Systems and convergence results

Table 4 and Figure 3 show the completed single-seed benchmark. Compiled LogRMS and fused AdamW had nearly identical median full-step latency: 107.369 versus 107.263 ms, a 0.10% difference. The LogRMS optimizer call itself was 15.2% slower, but forward/backward computation dominated the complete step. LogRMS reduced optimizer-call latency by 52.5% relative to foreach AdamW and completed the full run 5.1% sooner. Relative to the tested torchao AdamW8bit path, LogRMS used 45.6% less total wall time and delivered 1.84× end-to-end token throughput.

The total LogRMS wall time was 226.2 s (3.5%) longer than fused AdamW. Its first update took 207.3 s versus 0.16 s for fused AdamW. This compilation delay accounts for 91.6% of the total runtime gap. The run produced two compiled graphs, no graph breaks, and no compilation failures. Inductor reported that it could not use CUDA Graph replay because optimizer state is updated in place; compilation remained active. Longer runs would spread the startup cost over more updates.

The exact state audit measured 396,120,800 bytes for AdamW and 99,030,016 bytes for LogRMS, a saving of 297,090,784 bytes (75%). Peak live PyTorch allocation was 2,290,672,128 bytes for fused AdamW and 2,000,312,320 bytes for LogRMS, a 12.7% reduction; peak reserved memory was 2,495,610,880 and 2,260,729,856 bytes, respectively. These whole-process peaks include the model, gradients, activations, temporary tensors, compiler

Table 3: Two-epoch TinyStories result (one final-run seed). Lower loss and perplexity are better. State is persistent tensor payload.

Optimizer	State B/param	Final val. loss	Test loss	Test perplexity	Wall time (s)
AdamW, FP32 moments	8.0001	1.5824	1.5475	4.6999	2,402.7
torchao AdamW 8-bit	2.0604	**1.5698**	**1.5352**	**4.6423**	6,727.6
LogRMS eager	**2.0000**	1.5780	1.5434	4.6807	12,570.3

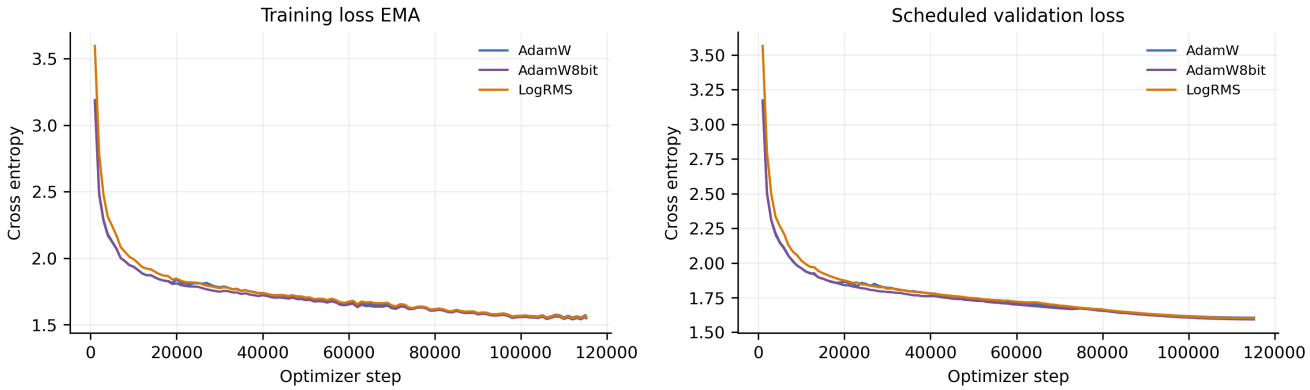


Figure 1: Training-loss EMA and scheduled validation trajectories for the final TinyStories runs. Curves are close after early training. The figure is useful as a trajectory check, not a substitute for multi-seed uncertainty.

buffers, and evaluation. They do not isolate optimizer memory. The state audit, not the CUDA peak, supports the two-byte claim.

Convergence remained stable but did not match AdamW under the shared policy. LogRMS final full-validation loss was 1.3708 versus 1.3510 for fused AdamW; test perplexity was 3.7698 versus 3.6911, a 2.13% gap. On the fixed scheduled validation subset, the loss difference narrowed from 0.1170 at the first evaluation to 0.0193 at the final evaluation. LogRMS may need different tuning, but this single run cannot show whether tuning would close the remaining gap. The result is stable training with a modest accuracy trade-off, not convergence parity.

7 Diagnostic objectives and failure probes

Small objectives were used to search for qualitative failures, not to rank optimizers globally. Five-seed saturated classification slightly favored LogRMS (99.56% versus 99.32%), but the ceiling makes the difference uninformative. On a nonlinear regression task, best tested mean MSE was 1.150 for LogRMS versus 0.407 for AdamW. On an ill-conditioned quadratic, both reached objective below 10^{-6} for every tested seed.

The clearest negative result was long-horizon two-dimensional Rosenbrock optimization: AdamW ended at mean objective 0.000511 while LogRMS ended at 0.706883. Only AdamW crossed the tested 0.5, 0.1, 0.01, and 0.001 thresholds in all seeds. The missing first moment may matter on curved, coupled objectives. This result cautions against drawing broad conclusions from language-model loss alone.

In a rare-gradient-spike probe, maximum update relative to baseline was 13.86 for LogRMS and 4.11 for AdamW; six-step parameter-path length was 1.533 versus 0.621. LogRMS's external global clipping remains an important conservative policy. Conversely, exact zero gradients produce no LogRMS parameter motion (apart from weight decay), whereas Adam's first moment can continue an update. That difference may be helpful or harmful depending on sparsity structure.

An eager CPU timing at 262,144 parameters measured 8.866 ms per LogRMS optimizer step and 0.733 ms for AdamW. At four million parameters, persistent payload was 7.63 MiB for LogRMS versus 30.52 MiB for AdamW, yet incremental peak allocation was 525.77 versus 133.41 MiB. These allocator-sensitive figures do not predict GPU capacity; they motivated compilation before making peak-memory claims.

8 Discussion

8.1 Why the design can work

RMS normalization gives each coordinate its own scale. A logarithmic code suits the second moment because that value can span many orders of magnitude. Stochastic rounding keeps small changes from being lost repeatedly when the state is stored. Sign history then reduces updates when recent gradients alternate often; it never increases

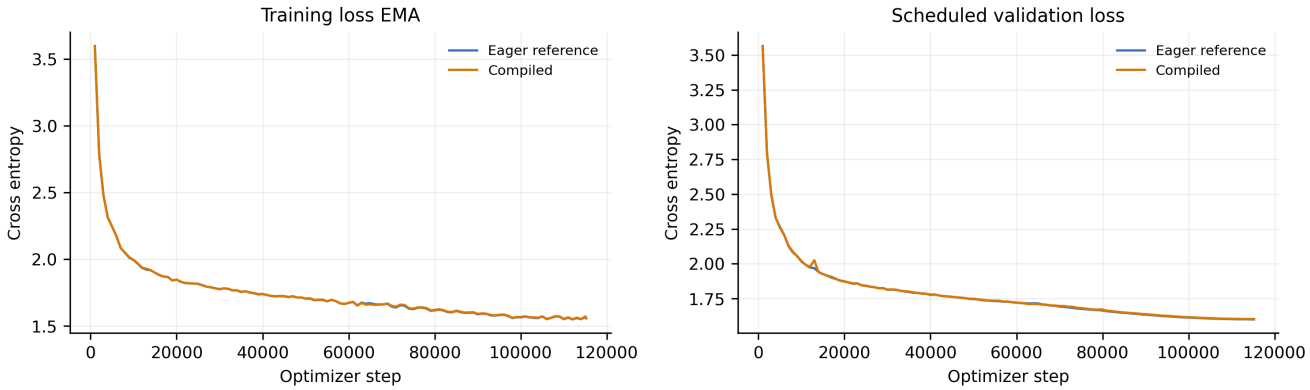


Figure 2: Eager reference trajectory and an execution-only `torch.compile` group implementation. Their close agreement shows that compilation preserved training behavior in this run.

Table 4: 49.5M-parameter TinyStories systems benchmark on one NVIDIA L4. One seed and a shared, untuned peak learning rate of 10^{-3} were used. Steady times exclude the first 20 updates. State is exact persistent tensor storage; lower loss, perplexity, and time are better.

Optimizer	State (B/param)	Full step (ms)	Opt. step (ms)	Total wall (min)	Final val. loss	Test loss	Test PPL
AdamW fused	8.0000	**107.263**	**7.609**	**106.52**	1.3510	**1.3059**	**3.6911**
LogRMS compiled	**2.0000**	107.369	8.768	110.29	1.3708	1.3270	3.7698
AdamW foreach	8.0000	117.328	18.439	116.17	**1.3506**	1.3061	3.6916
torchao AdamW8bit	2.0460	206.205	113.975	202.82	1.3570	1.3125	3.7156

them.

The experiments show that this combination can train the tested models, but do not prove that sign history always helps. The repository includes an RMS-only setting for a direct comparison in future studies.

8.2 Tuning policy

LogRMS may not work best with AdamW's hyperparameters. A fair comparison should give each optimizer the same tuning *budget*, but does not need to force the same values. The current defaults are $\beta_2 = 0.999$, $\epsilon = 10^{-8}$, stochastic rounding, and global norm clipping. Learning rate and weight decay still need to be tuned for each task. The current results do not support a fixed learning-rate multiplier relative to AdamW.

8.3 Where the memory benefit matters

The saving matters most when optimizer state takes a large share of training memory. Examples include full-parameter training with small microbatches, activation checkpointing, replicated state, or CPU offload. ZeRO and FSDP divide the saving among workers, but do not remove it. Smaller state can also reduce checkpoint size and state traffic.

Compared with 8-bit Adam, the state-size difference is small. Runtime and convergence then matter more. LogRMS was much faster than the tested torchao AdamW8bit implementation, but that result applies only to this software and hardware setup.

9 Limitations and future work

The transformer comparisons use one final-run seed, so they do not establish optimizer equivalence. The larger experiment is still a 49.5M-parameter model on TinyStories; it does not represent long-context, large-batch, sparse, multimodal, or distributed training. The 49.5M study also used a shared learning rate schedule instead of tuning each optimizer separately. The CUDA memory peaks cover the whole process rather than the optimizer alone. Finally, compilation was tested on one NVIDIA L4 software stack and took 207 seconds on the first update.

The next study should compare independently tuned AdamW, 8-bit AdamW, LogRMS, and the all-ones RMS ablation over at least three held-out seeds on the 49.5M transformer or a larger model. Initialization, data order, token count, tuning budget, clipping, weight decay, and evaluation tokens should be matched. Required measurements include loss versus tokens and wall time, peak allocated and reserved GPU memory, checkpoint size, log-code saturation, history readiness, gradient and update norms, and nonfinite events.

A dedicated memory test should record live allocation before, during, and after the optimizer update, audit the stored optimizer tensors separately, and record driver-level process memory. Repeating the test at several model sizes would separate per-parameter memory from fixed compiler and allocator costs.

Future kernels should preserve the current update rule while improving CUDA, Triton, and JAX support. Sep-

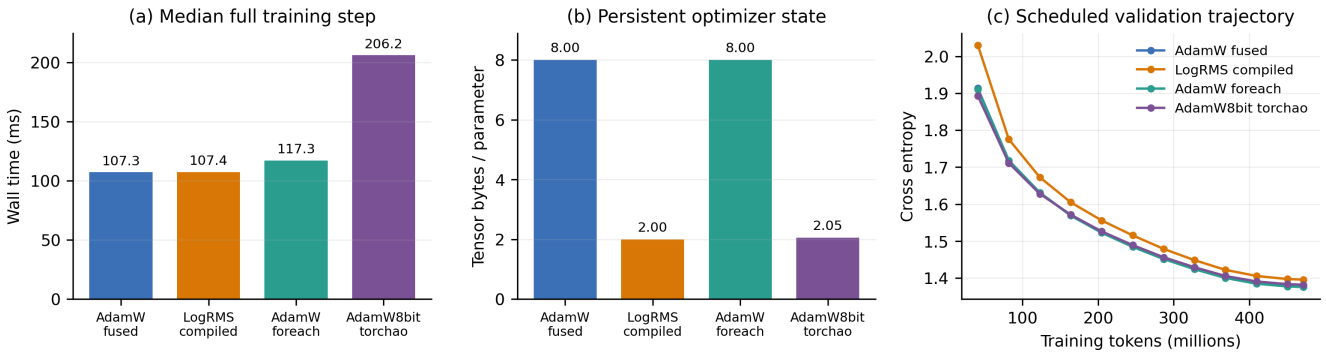


Figure 3: The 49.5M benchmark. Compiled LogRMS matches fused AdamW's median full-step latency while using one-quarter of its persistent tensor state. The validation gap narrows with training but remains nonzero at the end of the shared, untuned schedule.

arate experiments should study spike handling, curved objectives, and whether a small momentum signal is worth its state cost.

10 Conclusion

LogRMS uses exactly two persistent tensor bytes per parameter and trained both 5.06M- and 49.5M-parameter TinyStories models without instability. On the 49.5M model, its compiled full-step time was within 0.10% of fused AdamW while using one-quarter of AdamW's persistent tensor state. LogRMS finished with 2.13% higher test perplexity under the shared, untuned schedule and required a 207-second first-step compilation.

These results show that the current LogRMS design can combine small optimizer state with practical training speed. They do not yet show equal convergence across tasks. Multi-seed tuning, distributed training, more model families, and optimizer-specific memory measurements are still needed.

Reproducibility statement

The release contains the current reference source with SHA-256 `097fa598...f0b0b7c`, integrity and semantic tests, exact algorithm documentation, compact CSV tables, raw Colab output transcripts, figures, and paper source. The 49.5M evidence included with this revision contains the per-run summary, scheduled evaluations, and checkpoint timings; the benchmark notebook separately emits per-step timing CSVs and an environment manifest. The optimizer is Apache-2.0 licensed. The paper identifies the single-seed scope and retains negative results. A public release should add the final author list, repository URL, environment lockfile, and an independently rerunnable multi-seed training script.

References

[1] Rohan Anil, Vineet Gupta, Tomer Koren, and Yoram Singer. Memory-efficient adaptive optimization. *arXiv preprint arXiv:1901.11150*, 2019.

[2] Jason Ansel et al. Pytorch 2: Faster machine learning through dynamic python bytecode transformation and graph compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pages 929–947, 2024.

[3] Jeremy Bernstein, Jiawei Zhao, Kamyar Azizzadenesheli, and Anima Anandkumar. signsgd with majority vote is communication efficient and fault tolerant. *arXiv preprint arXiv:1810.05291*, 2018.

[4] Xiangning Chen, Chen Liang, Da Huang, Esteban Real, Kaiyuan Wang, Hieu Pham, Xuanyi Dong, Thang Luong, Cho-Jui Hsieh, Yifeng Lu, and Quoc V. Le. Symbolic discovery of optimization algorithms. *arXiv preprint arXiv:2302.06675*, 2023.

[5] Tim Dettmers, Mike Lewis, Sam Shleifer, and Luke Zettlemoyer. 8-bit optimizers via block-wise quantization. *arXiv preprint arXiv:2110.02861*, 2021.

[6] John Duchi, Elad Hazan, and Yoram Singer. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, 12:2121–2159, 2011.

[7] Ronen Eldan and Yuanzhi Li. Tinystories: How small can language models be and still speak coherent english? *arXiv preprint arXiv:2305.07759*, 2023.

[8] Suyog Gupta, Ankur Agrawal, Kailash Gopalakrishnan, and Pritish Narayanan. Deep learning with limited numerical precision. In *Proceedings of the 32nd International Conference on Machine Learning*, 2015.

[9] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

[10] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.

[11] Noam Shazeer and Mitchell Stern. Adafactor: Adaptive learning rates with sublinear memory cost. In *Proceedings of the 35th International Conference on Machine Learning*, 2018.